

Here is the step-by-step logical flow of data and user interaction through your MERN stack architecture.

1. The Onboarding & Authentication Flow (The Gateway)

The journey begins with separating core authentication from heavy profile data, keeping the initial load lightning fast.

- **Sign Up/Login:** The user enters via the client. The React app sends credentials to the `/api/users/auth` route.
- **Token Generation:** The Node.js server validates the credentials and returns an HTTP-only cookie containing a JWT (JSON Web Token) for secure, stateless session management.
- **The "Premium" Onboarding:** Instead of one massive form, the user experiences a multi-step onboarding wizard.
 - *Step A:* They select their role (Student or Alumni).
 - *Step B:* They input their professional details (Skills, Company, and Graduation Year).
 - *Backend Action:* The server creates the lightweight User document first, then links it to a detailed Profile document.

2. The Modular Dashboard (The Core Hub)

Once authenticated, the React router directs the user to a dashboard tailored to their specific role, pulling data from multiple independent micro-services (or modular controllers).

- **For a Student:** The dashboard prioritizes the "Mentorship" and "Jobs" modules. It fetches suggested alumni based on matching skills arrays.
- **For an Alumni:** The dashboard prioritizes "Mentorship Requests" (inbound) and "Events."
- **State Management:** The global store (e.g., Zustand/Redux) holds the basic user identity, while React Query (or similar tools) fetches and caches the dashboard widgets asynchronously so the UI never blocks.

3. The Mentorship Lifecycle (Transactional Flow)

This flow connects two users and transitions them from strangers to active communicators.

- **Discovery:** A student filters the Alumni directory. The client sends a query to the server (`/api/profiles?skills=React&company=Google`).
- **The Request:** The student clicks "Request Mentorship." The client sends a payload to the Mentorship module. The server creates a Mentorship document with a status: `'pending'`.
- **The Notification:** The server triggers a notification (potentially using Redis Pub/Sub to push an alert to the Alumni's active session).

- **The Handshake:** The Alumni reviews the request and clicks "Accept." The server updates the status to 'accepted' and automatically generates a new Conversation document in the Messaging module, linking both userIds.

4. The Job Referral Pipeline (Time-Sensitive Flow)

This flow involves posting and interacting with expiring data.

- **Posting:** An alumni creates a job referral. The data is saved to the Jobs collection with a TTL (Time-To-Live) index so the database automatically deletes the document after 90 days, keeping the database perfectly tidy without manual cron jobs.
- **Interaction:** When a user clicks "Apply/Request Referral" on a job card, the system intercepts the action. Instead of a complex internal application system, it triggers an automated direct message (using the Messaging module) from the applicant to the poster with a pre-filled template referencing the jobId.

5. Real-Time Messaging Flow (The High-Traffic Engine)

Because standard HTTP requests are too slow for chat, this flow breaks away from the standard REST API.

- **The Connection:** Upon login, the React client establishes a persistent WebSocket connection (via Socket.io) with the Node server.
- **Sending a Message:** When User A types a message, it is emitted via the WebSocket.
- **Processing:** The Node server receives the socket event, immediately saves the Message document to MongoDB, and updates the lastMessageAt timestamp on the parent Conversation.
- **Broadcasting:** The server checks if User B has an active WebSocket connection. If yes, it pushes the message directly to User B's screen instantly. If no, it increments an unread badge counter in the database for User B's next login.

The transition from a "pending mentorship request" to an "active real-time websocket chat" is usually the trickiest part of the user journey to engineer perfectly.